

OTIMIZAÇÃO DE PROCESSOS DE COLETA EM CENTROS DE DISTRIBUIÇÃO: UMA ABORDAGEM VIA GRAFOS TRIPARTIDOS

2023009646 - Rafael Lemes Scarpel

2022000969 - Tiago Antonio Vilela Carvalho

2021001120 - Breno Yukihiro Hirakawa

2023005342 - Enzo Andrade Junqueira Lopes

CMAC03 - ALGORITMOS EM GRAFOS

Prof. Rafael Frinhani



INSTITUTO DE
MATEMÁTICA E
COMPUTAÇÃO

UNIFEI - Itajubá



Otimização de Processos de Coleta em Centros de Distribuição: Uma abordagem via grafos tripartidos

1 Introdução

1.1 Objetivos do Projeto

Este projeto tem como objetivo desenvolver uma solução computacional para o problema de seleção ótima de pedidos em waves, aplicado ao contexto de centros de distribuição de grandes plataformas de comércio eletrônico. O problema consiste em selecionar, a cada etapa operacional, um subconjunto de pedidos do backlog a ser processado simultaneamente, de forma a maximizar a produtividade do processo de coleta de itens no armazém.

A produtividade é medida pela razão entre o total de unidades de itens coletadas e o número de corredores visitados para realizar essa coleta. O principal desafio é identificar, dentre os pedidos disponíveis, aqueles que, quando agrupados, permitam concentrar a coleta na menor quantidade possível de corredores, reduzindo deslocamentos e aumentando a eficiência operacional. Para isso, a proposta do projeto é modelar as relações entre pedidos, itens e corredores por meio de um grafo tripartido.

Os objetivos específicos do projeto são: (i) Compreender e delimitar o problema logístico de coleta em waves, identificando seus elementos, restrições e função objetivo; (ii) Modelar formalmente o problema por meio de um grafo tripartido ponderado, representando as relações entre pedidos, itens e corredores com suas respectivas quantidades; (iii) Desenvolver e implementar um protótipo de software que, dado um conjunto de pedidos no backlog e a disponibilidade de itens por corredor, produza uma wave viável; (iv) Validar as soluções produzidas utilizando o verificador fornecido e avaliar o desempenho do algoritmo em instâncias de diferentes tamanhos e características.

2 Trabalhos relacionados

Uma das abordagens de interesse utiliza a teoria dos grafos para representar o ambiente do centro de distribuição na resolução do problema de coleta. O trabalho de Çelik e Süral (2019) - Order picking in parallel-aisle warehouses with multiple blocks: complexity and a graph theory-based heuristic apresenta uma heurística baseada em grafos para o problema de seleção ótima em armazéns com múltiplos blocos, representando corredores, interseções e posições de coleta como elementos de um grafo. Sua modelagem motivou a representação adotada neste trabalho, na qual pedidos, itens e corredores são organizados em três conjuntos distintos de vértices.

Outra linha de pesquisa amplamente explorada consiste

na utilização de heurísticas construtivas inspiradas no problema clássico de Set Cover. No trabalho de Václav Chvátal (1979) é proposto uma heurística gulosa para o problema de cobertura de conjuntos (Set Covering Problem), baseada na seleção iterativa do subconjunto que apresenta a melhor relação entre o número de elementos ainda não cobertos e o custo de sua inclusão. Neste trabalho, essa ideia é adaptada ao problema de seleção de waves, de forma que a construção da wave é realizada de forma gulosa, priorizando a inclusão dos pedidos que proporcionam o maior ganho em quantidade de pedidos atendidos em relação ao aumento do número de corredores visitados.

3 Metodologia

Nesta seção, é apresentado o percurso adotado para a implementação da solução computacional proposta, passando pela definição das variáveis e restrições, a natureza dos dados de entrada, a construção do modelo de grafos e dos algoritmos.

3.1 Detalhamento do Problema

O problema de seleção ótima de waves consiste em determinar, a partir de um conjunto de pedidos pendentes (*backlog*), um subconjunto de pedidos que será processado simultaneamente, denominado *wave*. Cada pedido é composto por um conjunto de itens com quantidades específicas, enquanto cada item pode estar disponível em um ou mais corredores do armazém, também com quantidades limitadas. A escolha adequada da *wave* influencia diretamente a eficiência da operação de coleta, pois pedidos que compartilham itens armazenados em poucos corredores reduzem o deslocamento dos operadores e aumentam a produtividade do processo.

Formalmente, considera-se o conjunto de pedidos O , o conjunto de itens I e o conjunto de corredores A . Para cada pedido $o \in O$, existe um subconjunto de itens I_o e, para cada item $i \in I$, um conjunto de corredores A_i onde esse item está disponível. As constantes u_{oi} e u_{ai} representam, respectivamente, a quantidade do item i requerida pelo pedido o e a quantidade disponível desse item no corredor a . Além disso, são definidos os limites inferior (LB) e superior (UB) para o número total de unidades de itens que podem compor uma *wave*.

O objetivo do problema é selecionar uma *wave* $O' \subseteq O$ e um conjunto de corredores $A' \subseteq A$ capazes de atender todos os pedidos escolhidos, maximizando a produtividade da coleta. Essa produtividade é medida pela razão entre o número total de unidades de itens coletadas e o número de corredores visitados, conforme a Equação (1):

$$\max \frac{\sum_{o \in O'} \sum_{i \in I_o} u_{oi}}{|A'|}. \quad (1)$$

A solução deve satisfazer três grupos de restrições. Primeiramente, o número total de unidades de itens pertencentes aos pedidos selecionados deve permanecer entre os limites LB e UB , garantindo que a *wave* possua tamanho operacionalmente viável:

$$LB \leq \sum_{o \in O'} \sum_{i \in I_o} u_{oi} \leq UB. \quad (2)$$

Além disso, para cada item presente na *wave*, os corredores selecionados devem possuir estoque suficiente para atender integralmente à demanda:

$$\sum_{o \in O'} u_{oi} \leq \sum_{a \in A'} u_{ai}, \quad \forall i \in I. \quad (3)$$

Apenas pares formados por um subconjunto de pedidos e um subconjunto de corredores que satisfaçam simultaneamente essas restrições são considerados soluções viáveis.

3.2 Modelagem

As instâncias do problema foram fornecidas em arquivos *.txt*. A primeira linha de cada dataset define a dimensão do cenário por meio de três números inteiros: o número total de pedidos no backlog, a quantidade de itens únicos e o número de corredores disponíveis. Na sequência, o arquivo apresenta um bloco de linhas com a composição de cada pedido, iniciando com a quantidade de itens distintos solicitados, seguida com pares de valores que são correspondentes ao ID do item, e sua demanda em unidades. O próximo bloco mapeia o estoque do armazém, listando os itens e suas quantidades disponíveis em cada corredor. Por fim, a última linha estipula os parâmetros operacionais da *wave*, definindo os limites inferior (LB) e superior (UB) de unidades totais de itens que podem ser agrupadas na solução.

Para a estrutura de representação do problema, foi definida a utilização de um grafo tripartido. Essa escolha foi motivada pelas relações existentes entre os três conjuntos envolvidos: Pedidos, Itens e Corredores. A partir dessa modelagem, é possível capturar, de maneira simultânea, a dependência de composição dos pedidos, em que cada pedido requer determinados itens, e a dependência de disponibilidade, visto que um item pode estar presente em determinados corredores.

Modelos baseados em grafos simples ou bipartidos também foram considerados, mas seriam capazes de capturar apenas uma das relações por vez, dificultando a visualização e compreensão da estrutura global do problema. O grafo tripartido, por sua vez, permite perceber e raciocinar as duas relações existentes em conjunto. Fica observável, por exemplo, que selecionar um pedido implica ativar um subconjunto de itens, o que, por sua vez, implica na necessidade de visitar ao menos um corredor que os contenha. Dessa forma, o problema é representado de uma maneira mais intuitiva e mais semelhante a disposição real dos itens dentro do armazém.

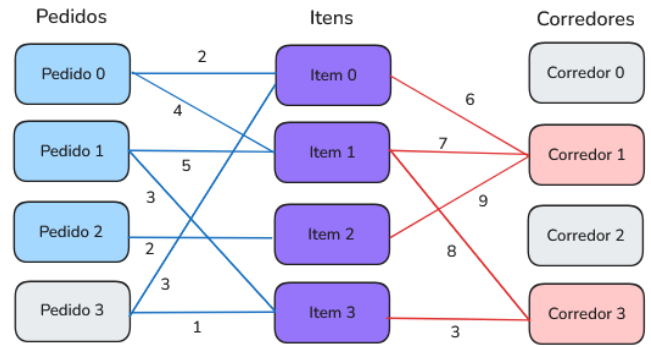


Figura 1: Grafo tripartido representando as relações entre pedidos, itens e corredores

O grafo é composto por três camadas de nós: à esquerda, os pedidos do backlog; Ao centro, os itens que compõem esses pedidos; E à direita, os corredores do armazém onde os itens estão armazenados. As arestas entre o conjunto de pedidos e o conjunto de itens indicam que aquele pedido solicita aquele item, e o seu peso indica a quantidade demandada. As arestas entre o conjunto de itens e a camada de corredores indicam que aquele item está disponível naquele corredor, com peso indicando a quantidade disponível. A coloração dos nós distingue os elementos selecionados para uma solução ótima: nós azuis representam os pedidos incluídos na *wave* (pedidos 0, 1, 2); o nó cinza à esquerda representa o pedido excluído da *wave* (pedido 3); nós vermelhos representam os corredores visitados (corredores 1 e 3); e nós cinza à direita representam os corredores não visitados (corredores 0, 2). As arestas em azul conectam pedidos selecionados aos itens que eles demandam. As arestas em vermelho conectam itens aos corredores visitados que os abastecem.

3.3 Desenvolvimento

O protótipo foi implementado na linguagem Python, adotando a biblioteca NetworkX para instanciar e manipular o cenário de acordo com a modelagem proposta na seção anterior. A principal solução proposta se baseia em um algoritmo guloso de cobertura de conjuntos, projetada para operar sobre o grafo tripartido (Pedidos $\rightarrow$ Itens $\rightarrow$ Corredores). Para maximizar a função objetivo e evitar que o algoritmo fique preso em ótimos locais inviáveis, implementou-se uma estratégia de múltiplas sementes, ou "multi-start". Essa abordagem testa diferentes itens de alta demanda como ponto de partida, construindo e expandindo as *waves* de forma iterativa, respeitando as limitações de volume e disponibilidade real de estoque no armazém.

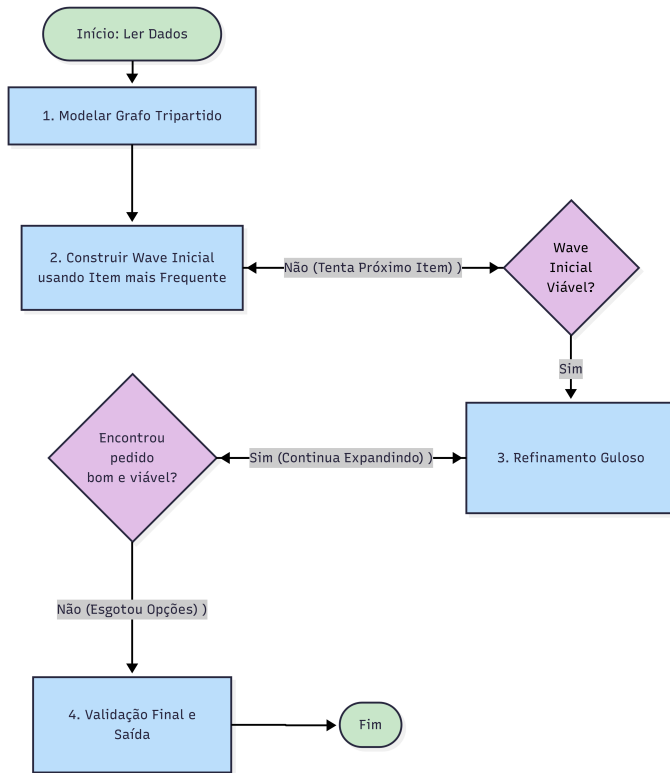


Figura 2: Fluxograma que ilustra a lógica da solução

O fluxograma ilustra a visão macro da solução desenvolvida. A primeira etapa para aplicar essa solução em python foi a tradução dos dados de entrada para a estrutura de dados escolhida. O Algoritmo 1 detalha esse processo de modelagem, em que foram utilizadas as funções da biblioteca *NetworkX* para tornar cada entidade em um nó alocado a seu respectivo conjunto. As restrições do problema são alocadas através das arestas direcionadas: as conexões entre pedidos e itens carregam o peso da demanda, enquanto as conexões entre itens e corretores carregam o peso da disponibilidade de estoque do armazém.

Algorithm 1: Modelagem do Grafo Tripartido Direcionado

Input: Arquivo texto contendo limites da *wave*, demandas de pedidos e estoques dos corretores

Output: Grafo Tripartido G , Limites *waveMin* e *waveMax*

```

1  $G \leftarrow$  Novo Grafo Direcionado vazio;
2  $waveMin, waveMax \leftarrow$ 
  LerLimitesOperacionais(Arquivo);
3 // Instanciação da camada de Pedidos e
  conexões de Demanda
4 for cada pedido  $\in$  Arquivo do
5   AdicionarNo( $G, id = pedido, camada = 'pedido'$ );
6   for cada (item, quantidade)  $\in$  pedido do
7     AdicionarNo( $G, id = item, camada = 'item'$ );
8     AdicionarAresta( $G, origem = pedido, destino =$ 
       item, peso = quantidade);
9   end
10 end
11 // Instanciação da camada de Corredores e
   conexões de Estoque
12 for cada corredor  $\in$  Arquivo do
13   AdicionarNo( $G, id = corredor, camada =$ 
     'corredor');
14   for cada (item, quantidade)  $\in$  corredor do
15     // O nó do item correspondente é
       referenciado na aresta
16     AdicionarAresta( $G, origem = item, destino =$ 
       corredor, peso = quantidade);
17   end
18 end
19 return  $G, waveMin, waveMax$ ;

```

Com o grafo instanciado, o algoritmo avança para a segunda fase, que consiste na formação do núcleo da wave. Para evitar uma busca exaustiva por todas as combinações de pedidos, é adotada uma abordagem de "múltiplas sementes". O grafo é consultado para identificar o item com maior grau de entrada, que corresponde ao item mais frequentemente requisitado pelos pedidos. Esse item atua como semente para um agrupamento inicial: o algoritmo pré-seleciona todos os pedidos que o contêm. Caso a soma das unidades desse núcleo exceda o limite superior de volume, ou o estoque global do armazém seja insuficiente para a demanda gerada, o agrupamento é descartado e o processo reinicia utilizando o próximo item mais frequente como semente.

Para a terceira fase, o algoritmo busca otimizar a eficiência da wave explorando o backlog de pedidos restantes. A cada iteração, os candidatos são avaliados e ranqueados por meio do Algoritmo 2, que prioriza os pedidos que tem itens armazenados em corretores já ativados na rota atual. O pedido com a melhor pontuação passa por uma checagem de disponibilidade de estoque e, caso sua inclusão aumente o valor da função objetivo, ou o agrupamento ainda precise atingir o limite mínimo, ele é incorporado ao subconjunto. Esse ciclo se repete até que o limite máximo de volume seja alcançado ou até que nenhum candidato adicional consiga elevar a produtividade do sistema.

Algorithm 2: Avaliação e Seleção Gulosa de Pedidos

Input: Grafo Tripartido G , Wave Atual W , Corredores Ativos S , Limite $waveMax$
Output: Melhor Pedido Viável ou Nulo

```

1  $Candidatos \leftarrow \emptyset$ ;
2  $VolAtual \leftarrow \text{Volume}(W, G)$ ;
3 // Fase A: Calcula o score de proximidade
  para os pedidos pendentes
4 for cada pedido  $\notin W$  do
5    $volPedido \leftarrow \text{Volume}(\text{pedido}, G)$ ;
6   if  $VolAtual + volPedido > waveMax$  then
7     continue;
8   ; // Ignora se ultrapassar o limite superior
9   end
10   $cobertos \leftarrow 0$ ;
11   $novos \leftarrow 0$ ;
12  for cada item  $\in \text{Sucessores}(\text{pedido}, G)$  do
13     $\text{CorredoresDoItem} \leftarrow \text{Sucessores}(\text{item}, G)$ ;
14    // Operação de conjuntos para checar
    se o corredor já está ativo
15    if  $\text{CorredoresDoItem} \cap S \neq \emptyset$  then
16       $cobertos \leftarrow cobertos + 1$ ;
17    else
18       $novos \leftarrow novos + 1$ ;
19    end
20  end
21   $score \leftarrow cobertos - novos$ ;
22   $Candidatos \leftarrow Candidatos \cup \{(score, \text{pedido})\}$ ;
23 end
24 // Fase B: Ordena os candidatos de forma
  decrescente pelo score
25  $\text{OrdenarDecrescente}(Candidatos, \text{chave}=score)$ ;
26 // Fase C: Retorna o primeiro candidato
  que não viola o estoque global
27 for cada  $(score, \text{candidato}) \in Candidatos$  do
28    $NovaDemanda \leftarrow$ 
     $\text{CalcularDemanda}(W \cup \{\text{candidato}\}, G)$ ;
29   if  $\text{VerificaEstoqueDisponivelNoGrafo}(G, NovaDemanda)$ 
    then
30     return candidato;
31   end
32 end
33 return Nulo;
34 ; // Nenhum pedido viável encontrado

```

Por fim, na última etapa, a solução candidata passa pela validação final. O algoritmo confere se o volume acumulado pela wave atingiu o limite mínimo exigido, e percorre as arestas do grafo pra assegurar que o subconjunto de corredores ativados possui as quantidades exatas para suprir todos os itens da demanda. Se a solução for considerada viável, os resultados são exportados.

Para estabelecer uma base de validação e comparação de desempenho, a arquitetura do software foi feita de forma modular, incluindo duas variações da abordagem principal apresentada. O motor central do algoritmo (`executaWave`) foi construído para receber funções dinâmicas como parâmetros,

permitindo a execução das três abordagens distintas de implementação para a mesma heurística gulosa:

- **Abordagem Principal:** A solução proposta neste projeto. Desde a identificação do item mais frequente até a validação de cobertura de estoque, o fluxo de dados se mantém fiel à modelagem proposta em grafos. Essa abordagem exige maior alocação de recursos da biblioteca *NetworkX*, mas é a que melhor traduz o problema de roteamento e alocação.
- **Abordagem com Dicionários:** Nessa variação, o uso de grafos foi totalmente suprimido. A representação do cenário, a busca pelo pedido inicial, o ranqueamento de candidatos e a resolução do problema de *Set Cover* foram executados utilizando apenas estruturas de dados nativas em Python: (*dicts* e *lists*). Serve como parâmetro de controle para avaliar o tempo de execução.
- **Abordagem Híbrida (Grafos e Dicionários):** Uma solução intermediária, em que a modelagem dos pedidos e o cálculo de *scores* por ocorrem na estrutura de grafos, aproveitando a facilidade de navegação por sucessores. Entretanto, no momento de definir a cobertura de corredores (*Set Cover*), o algoritmo converte a demanda e volta para o uso de dicionários.

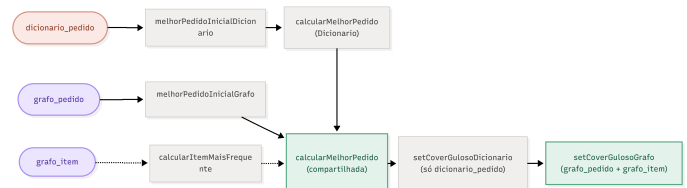


Figura 3: Fluxograma que ilustra a estrutura da main.py

A Figura 3 apresenta o diagrama da estrutura implementada no módulo principal (`main.py`), demonstrando como os dados de entrada (à esquerda) são ramificados nas diferentes abordagens para a execução.

4 Resultados e Discussão

As três abordagens propostas foram avaliadas utilizando as vinte instâncias disponibilizadas para o problema, executando-se exatamente a mesma heurística gulosa em todas elas. A diferença entre os experimentos está exclusivamente na estrutura utilizada para representar os dados durante a execução: uma implementação baseada apenas em dicionários, uma implementação baseada no grafo tripartido e uma implementação centrada no grafo de itens.

Para cada execução foram registrados o número de unidades selecionadas para compor a wave, a quantidade de corredores utilizados para atender à demanda, o valor da função objetivo e o tempo total de processamento. Esses indicadores permitiram comparar simultaneamente a qualidade das soluções produzidas e o custo computacional de cada abordagem.

De maneira geral, a implementação baseada em dicionários apresentou os melhores valores para a função objetivo na maior parte das instâncias avaliadas, obtendo isoladamente o

melhor resultado em sete dos vinte casos analisados. Em outras cinco instâncias, seus resultados coincidiram com os obtidos pela abordagem baseada no grafo de pedidos, indicando que ambas frequentemente produzem soluções de qualidade semelhante.

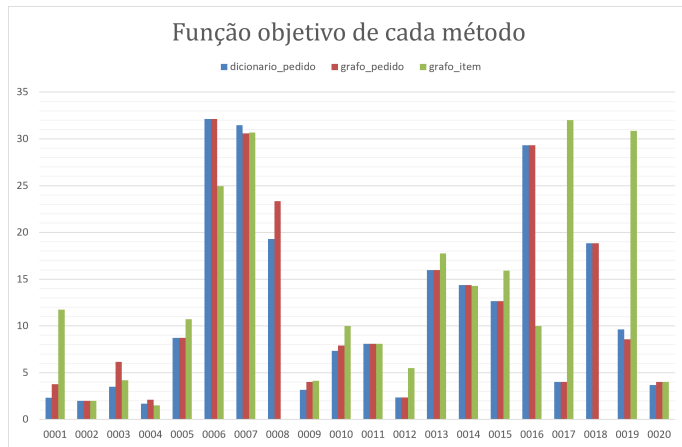


Figura 4: Gráfico que demonstra a diferença entre as funções objetivo

A implementação baseada no grafo de pedidos apresentou comportamento bastante próximo ao da versão baseada em dicionários em relação à qualidade das soluções, porém com maior tempo de execução na maioria dos casos. Em instâncias de maior porte, como as instâncias 0006, 0007, 0013, 0014 e 0015, observou-se que o custo adicional da manipulação da estrutura de grafos não foi compensado por ganhos significativos na função objetivo, produzindo resultados praticamente equivalentes aos obtidos pela implementação em dicionários.

Já a abordagem baseada no grafo de itens apresentou um comportamento distinto. Embora tenha produzido funções objetivo inferiores em diversas instâncias, essa implementação obteve, de forma consistente, os menores tempos de processamento para os datasets de maior dimensão. Nas instâncias 0013, 0014 e 0015, por exemplo, seu tempo de execução foi substancialmente inferior ao das demais abordagens, evidenciando melhor escalabilidade conforme aumenta o tamanho do problema, representado pela Figura 5.

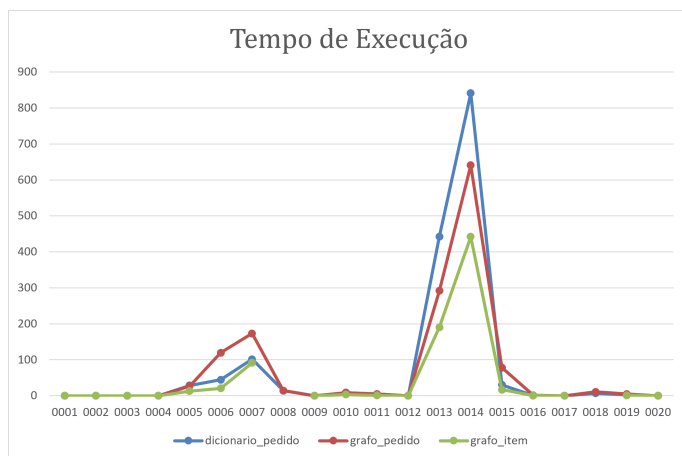


Figura 5: Gráfico representando a diferença de tempo entre as abordagens

Entretanto, essa redução no tempo computacional ocorreu com perda de robustez. Em duas instâncias (0008 e 0018), a implementação baseada no grafo de itens não conseguiu construir uma wave válida dentro das restrições impostas pelo problema, encerrando a execução com a indicação de "wave fora dos limites". Esse comportamento não foi observado nas abordagens baseadas em dicionários nem na modelagem por grafo de pedidos. É possível notar na figura 4 que as instâncias 0008 e 0018 não possuem dados.

Outra característica observada é que, nas instâncias de pequeno porte, as diferenças entre as três implementações foram pouco expressivas, tanto em relação ao valor da função objetivo quanto ao tempo de execução. À medida que o tamanho das instâncias aumenta, entretanto, tornam-se evidentes os impactos da estrutura de dados escolhida sobre o desempenho do algoritmo. Enquanto as implementações baseadas em dicionários e no grafo de pedidos tendem a preservar soluções de melhor qualidade, a implementação baseada no grafo de itens privilegia o desempenho computacional, tornando-se significativamente mais rápida em problemas maiores.

Esses resultados evidenciam um compromisso entre qualidade da solução e eficiência computacional. A abordagem baseada em dicionários mostrou-se mais consistente na obtenção das melhores funções objetivo, enquanto a implementação baseada no grafo de itens destacou-se pela rapidez e melhor escalabilidade. Já a implementação baseada no grafo de pedidos apresentou comportamento intermediário, reproduzindo resultados próximos aos obtidos pela versão em dicionários, porém com maior custo computacional em boa parte das instâncias analisadas.

5 Conclusões

Este trabalho apresentou uma abordagem heurística para o problema de formação de waves em centros de distribuição, utilizando diferentes estruturas de dados para representar as relações entre pedidos, itens e corredores. Foram implementadas e comparadas três estratégias distintas: uma baseada em dicionários, uma baseada em um grafo de pedidos e uma baseada em um grafo de itens, todas utilizando a mesma heurística gulosa para construção das waves e seleção dos corredores necessários para atender à demanda.

Os resultados demonstraram que a escolha da estrutura de dados exerce influência significativa tanto na qualidade das soluções quanto no desempenho computacional do algoritmo. A implementação baseada em dicionários apresentou, de forma geral, as melhores funções objetivo, mostrando-se a abordagem mais consistente na construção de waves com maior aproveitamento dos corredores disponíveis. Em diversas instâncias, seus resultados foram equivalentes aos obtidos pela implementação baseada em grafo de pedidos, indicando que ambas produzem soluções de qualidade semelhantes.

Porém, a implementação baseada em grafos de itens destacou-se pelo menor tempo de execução mesmo em instâncias de maior porte, o que evidencia uma melhor escalabilidade em relação às demais abordagens. Entretanto, este ganho no tempo de execução foi acompanhado de alguns fatores na qualidade das soluções, assim como instâncias inviáveis, indicando que esta representação prioriza a eficiência computacional em detrimento da robustez da solução.

A abordagem baseada no grafo de pedidos apresentou comportamento intermediário, produzindo resultados próximos aos da implementação baseada em dicionários, porém com maior custo computacional na maior parte dos experimentos realizados. Dessa forma, os resultados obtidos sugerem que o aumento da complexidade da estrutura de dados nem sempre se traduz em melhorias na qualidade da solução, sendo necessário considerar o equilíbrio entre desempenho e custo computacional durante a escolha da representação mais adequada.

Além da comparação entre as implementações, este trabalho demonstrou a viabilidade da modelagem do problema por meio de um grafo tripartido, representando explicitamente as relações entre pedidos, itens e corredores. Essa modelagem constitui uma alternativa às representações tradicionalmente encontradas na literatura e mostrou-se adequada para a aplicação de heurísticas gulosas na construção das waves.

Como trabalhos futuros, sugere-se o desenvolvimento de heurísticas mais sofisticadas para seleção dos pedidos, a incorporação de técnicas de busca local ou metaheurísticas para refinamento das soluções, bem como a investigação de diferentes critérios para escolha dos corredores e avaliação da função objetivo. Também é interessante explorar otimizações específicas para a implementação baseada em grafos, buscando reduzir seu custo computacional sem comprometer a qualidade das soluções obtidas.

Referências

